



DECEMBER 31, 2025

CLOUD COMPUTING

ENGR. WAQAS SALIM

ASSIGNMENT-2

HAJRA SARWAR
2023-BSE(5A)-022



ADVANCED TERRAFORM & NGINX MULTI-TIER ARCHITECTURE

Table of Contents

1. Executive Summary	2
• Brief overview of the assignment	2
• Infrastructure deployed	2
• Key achievements	2
2. Architecture Design	2
• Architecture diagram	2
• Component descriptions	2
• Network topology	3
• Security design	3
3. Implementation Details	4
• Part 1: Infrastructure setup	4
• Part 2: Webserver module	8
• Part 3: Server scripts	10
• Part 4: Deployment	11
• Part 5: Testing and verification	14
• Part 6: Cleanup	19
4. Testing Results	20
• Load balancing tests	20
• Cache performance tests	20
• High availability tests	20
• Security tests	20
• Performance metrics	20
5. Challenges & Solutions	21
• Problems encountered	21
• How you solved them	21
• Lessons learned	21
6. Conclusion	21
• Summary of work completed	21
• Skills acquired	21
• Future improvements	21
7. Appendices	21
• Complete code listings	21
• Configuration files	21
• Additional screenshots	21
• References	21

1. Executive Summary

This assignment focused on designing and deploying a **high-availability multi-tier web infrastructure** on AWS using **Terraform modules** and **Nginx** as a reverse proxy/load balancer. The objective was to demonstrate Infrastructure as Code best practices, advanced Nginx configurations (SSL, caching, load balancing), secure networking, and high availability with failover.

Infrastructure Deployed:

- **Nginx Server:** Acts as reverse proxy/load balancer with HTTPS, caching, and security headers.
- **Backend Web Servers:** Three EC2 instances (web-1, web-2 as primary, web-3 as backup) running Apache HTTP Server.
- **Networking:** Custom VPC, public subnet, internet gateway, route table.
- **Security:** Nginx and backend security groups with proper ingress/egress rules.
- **Automation:** Terraform modules for networking, security, and web servers; shell scripts for server setup.
- **Monitoring & Testing:** Load balancing verification, caching functionality, backup server failover, SSL/TLS validation.

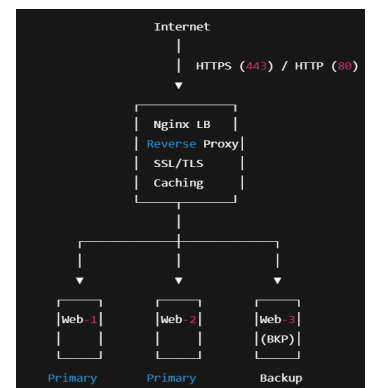
Key Achievements:

- Modular Terraform setup enabling reusability and scalability.
- Successful deployment of 4 EC2 instances with automated configuration.
- Nginx reverse proxy configured with **load balancing, caching, HTTPS, and security headers**.
- High availability tested using **primary server failure simulation**; backup server successfully activated.
- Documentation and scripts created to allow easy replication and maintenance.

2. Architecture Design

2.1 Architecture Diagram

The diagram shows the overall flow of the web infrastructure. Users access the system from the Internet via HTTP/HTTPS. Traffic first reaches the Nginx load balancer, which handles SSL, caching, and distributes requests to the backend servers. Web-1 and Web-2 serve as primary servers, while Web-3 acts as a backup for high availability. Security groups control access, allowing only necessary traffic to Nginx and the backend servers. This setup ensures secure, load-balanced, and highly available web service.



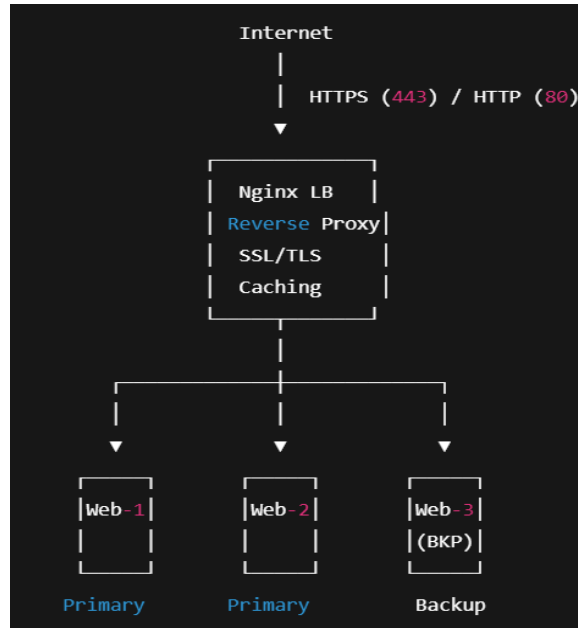
2.2 Component descriptions

- **Nginx Server (LB/Reverse Proxy):**
 - Distributes traffic to primary web servers using round-robin load balancing.
 - Serves HTTPS traffic with a self-signed certificate.
 - Caching implemented to improve performance (X-Cache-Status HIT/MISS).
 - Health check endpoint ensures backend availability.
 - Security headers: HSTS, X-Frame-Options, X-Content-Type-Options, X-XSS-Protection.
- **Backend Web Servers (Web-1, Web-2, Web-3):**
 - Apache HTTP server deployed via Terraform module.
 - Custom HTML page showing hostname, private/public IP, deployment timestamp, and status.
 - Web-1 and Web-2 are primary; Web-3 is backup for failover.
 - Scripts utilize IMDSv2 for secure metadata access.
- **Networking:**
 - **VPC** with custom CIDR block.
 - **Public subnet** with `map_public_ip_on_launch` enabled.
 - **Internet Gateway** attached for public connectivity.
 - **Route Table** associated with subnet for internet routing.
- **Security Design:**
 - **Nginx Security Group:** Ports 22 (SSH) restricted to your IP, 80/443 open to all.
 - **Backend Security Group:** Port 22 restricted to your IP, port 80 restricted to Nginx SG only.
 - All resources tagged with environment and project details for clarity.
- **High Availability:**
 - Backup web server automatically handles traffic if primary servers fail.

- Load balancing verified through repeated requests showing traffic alternation.
- Server health monitored via Nginx logs and optional bonus health check scripts.
- **Deployment Automation:**
 - Terraform modules for networking, security, and webserver configuration.
 - Scripts for Nginx and Apache ensure consistent, repeatable setup.
 - Outputs provided to guide manual updates for backend IPs in Nginx.

2.3 Network topology

The deployed infrastructure follows a **multi-tier architecture** with high availability and proper network segmentation. The topology is as follows:



Key Points:

- **VPC:** Single VPC with CIDR 10.0.0.0/16.
- **Subnet:** Public subnet 10.0.10.0/24 hosting all EC2 instances.
- **Internet Gateway:** Provides internet access to Nginx and backend servers.
- **Routing:** Route table attached to subnet, routes 0.0.0.0/0 via IGW.
- **High Availability:** Web-3 acts as a backup server, only serving traffic if Web-1 or Web-2 fail.

2.3 Security Design

Security is enforced at multiple layers using **AWS Security Groups** and **TLS/HTTPS**.

1. Nginx Security Group

- **Ingress Rules:**
 - SSH (22) → Only from your IP
 - HTTP (80) → Anywhere (0.0.0.0/0)
 - HTTPS (443) → Anywhere (0.0.0.0/0)
- **Egress:** All traffic allowed

2. Backend Security Group

- **Ingress Rules:**
 - SSH (22) → Only from your IP
 - HTTP (80) → Only from Nginx security group
- **Egress:** All traffic allowed

Additional Measures:

- **SSL/TLS:** Self-signed certificate generated on Nginx, enabling HTTPS.
- **Security Headers:** Added Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options, X-XSS-Protection.
- **SSH Key Management:** EC2 key pairs generated per instance, private keys never committed.
- **IAM Principle:** Least privilege applied; only required resources have permissions.

3. Implementation Details

Part 1: Infrastructure setup

1.1 Project structure (create files & folders)

TASK:

1. Create all necessary files and directories

```
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> tree /F
Folder PATH listing for volume New Volume
Volume serial number is 2481-A0EA
D:.
.
  .gitignore
  locals.tf
  main.tf
  outputs.tf
  README.md
  terraform.tfvars
  variables.tf
docs
  architecture.md
  HajraSarwar-BSE(SA)-022-CC-Assignment2.docx
  troubleshooting.md
modules
  networking
    main.tf
    outputs.tf
    variables.tf
  security
    main.tf
    outputs.tf
    variables.tf
  webserver
    main.tf
    outputs.tf
    variables.tf
screenshots
  bonus
  part1
  part2
  part3
  part4
  part5
  part6
scripts
  apache-setup.sh
  nginx-setup.sh
```

2. Implement proper .gitignore to exclude sensitive files

```
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> type .gitignore
# Terraform
.terraform/
terraform.tfstate
terraform.tfstate.backup
*.tfvars

# SSH keys
*.pem
*.key

# OS files
.DS_Store
```

3. Document the project structure in README.md

README - Notepad

File Edit Format View Help

Project Structure

This Terraform project is organized to modularize infrastructure components for easier management and scalability

```
Assignment2/
├── main.tf # Root Terraform configuration
├── variables.tf # Input variables for the root module
├── outputs.tf # Output values from the root module
├── locals.tf # Local values for calculations and reusability
├── terraform.tfvars # Variable values for deployment
├── .gitignore # Files and directories to be ignored by Git
├── modules/ # Reusable Terraform modules
│   ├── networking/ # Networking module (VPC, subnets, routing)
```

1.2 Variable configuration

TASK:

1. Add validation rules for CIDR blocks
2. Add descriptions for all variables

Variables.tf:

```
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> type variables.tf
variable "vpc_cidr_block" {
  description = "CIDR block for the VPC"
  type        = string

  validation {
    condition     = can(cidrnetmask(var.vpc_cidr_block))
    error_message = "VPC CIDR block must be valid."
  }
}

variable "subnet_cidr_block" {
  description = "CIDR block for the public subnet"
  type        = string

  validation {
    condition     = can(cidrnetmask(var.subnet_cidr_block))
    error_message = "Subnet CIDR block must be valid."
  }
}

variable "availability_zone" {
  description = "Availability zone where resources will be deployed"
  type        = string
}

variable "env_prefix" {
  description = "Environment name prefix (e.g. dev, prod)"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}

variable "public_key" {
  description = "Path to public SSH key"
  type        = string
}

variable "private_key" {
  description = "Path to private SSH key"
  type        = string
}

variable "backend_servers" {
  description = "Backend server definitions"
  type        = list(object({
    name       = string
    script_path = string
  }))
}
```

3. Set appropriate defaults where applicable
4. Populate terraform.tfvars with your values

Terraform.tfvars:

```
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> type terraform.tfvars
vpc_cidr_block      = "10.0.0.0/16"
subnet_cidr_block   = "10.0.10.0/24"
availability_zone    = "me-central-1a"
env_prefix          = "prod"
instance_type       = "t3.micro"

public_key = "~/ssh/id_ed25519.pub"
private_key = "~/ssh/id_ed25519"

backend_servers = [
  {
    name       = "backend-1"
    script_path = "scripts/apache-setup.sh"
  }
]
```

1.3 Networking module

TASK:

1. Create VPC resource

```
# module.networking.aws_vpc.this will be created
resource "aws_vpc" "this" {
  arn                = (known after apply)
  cidr_block         = "10.0.0.0/16"
  default_network_acl_id = (known after apply)
  default_route_table_id = (known after apply)
  default_security_group_id = (known after apply)
  dhcp_options_id     = (known after apply)
  enable_dns_hostnames = (known after apply)
  enable_dns_support   = true
  enable_network_address_usage_metrics = (known after apply)
  id                  = (known after apply)
  instance_tenancy    = "default"
  ipv6_association_id = (known after apply)
  ipv6_cidr_block      = (known after apply)
  ipv6_cidr_block_network_border_group = (known after apply)
  main_route_table_id = (known after apply)
  owner_id             = (known after apply)
  region               = "me-central-1"
  tags                 = {
    "Name" = "prod-vpc"
  }
  tags_all             = {
    "Name" = "prod-vpc"
  }
}
```

2. Create subnet with map_public_ip_on_launch = true
3. Create and attach Internet Gateway

```
# module.networking.aws_internet_gateway.this will be created
+ resource "aws_internet_gateway" "this" {
+   arn          = (known after apply)
+   id           = (known after apply)
+   owner_id     = (known after apply)
+   region       = "me-central-1"
+   tags         = {
+     "Name" = "prod-igw"
+   }
+   tags_all     = {
+     "Name" = "prod-igw"
+   }
+   vpc_id       = (known after apply)
+ }
}
```

4. Configure routing table

```
# module.networking.aws_route_table.this will be created
+ resource "aws_route_table" "this" {
+   arn          = (known after apply)
+   id           = (known after apply)
+   owner_id     = (known after apply)
+   propagating_vgws = (known after apply)
+   region       = "me-central-1"
+   route        = [
+     {
+       cidr_block      = "0.0.0.0/0"
+       gateway_id      = (known after apply)
+       # (11 unchanged attributes hidden)
+     },
+   ]
+   tags         = {
+     "Name" = "prod-rt"
+   }
+   tags_all     = {
+     "Name" = "prod-rt"
+   }
+   vpc_id       = (known after apply)
+ }
}
```

5. Add proper tags to all resources using env_prefix

Outputs.tf:

```
PS D:\FJMU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2\modules\networking> type outputs.tf
output "vpc_id" {
  value = aws_vpc.this.id
}

output "subnet_id" {
  value = aws_subnet.this.id
}

output "igw_id" {
  value = aws_internet_gateway.this.id
}

output "route_table_id" {
  value = aws_route_table.this.id
}
```

main.tf:

```
PS D:\FJMU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2\modules\networking> type main.tf
resource "aws_vpc" "this" {
  cidr_block = var.vpc_cidr_block

  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}

resource "aws_subnet" "this" {
  vpc_id            = aws_vpc.this.id
  cidr_block        = var.subnet_cidr_block
  availability_zone  = var.availability_zone
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.env_prefix}-subnet"
  }
}

resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id

  tags = {
    Name = "${var.env_prefix}-igw"
  }
}

resource "aws_route_table" "this" {
  vpc_id = aws_vpc.this.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

resource "aws_route_table_association" "this" {
  subnet_id = aws_subnet.this.id
  route_table_id = aws_route_table.this.id
}
```

1.4 Security module

TASK:

1. Create Nginx security group with appropriate rules

```

# module.aws_security_group.nginx_sg will be created
resource "aws_security_group" "nginx_sg" {
  name               = "nginx_sg"
  description        = "Security group for Nginx"
  vpc_id             = var.vpc_id
  ingress            = [
    {
      cidr_blocks = [
        "0.0.0.0/0",
      ]
      from_port    = 0
      ipv6_cidr_blocks = []
      prefix_list_ids = []
      protocol     = "tcp"
      security_groups = []
      self         = false
      to_port      = 80
    },
    {
      cidr_blocks = [
        "0.0.0.0/0",
      ]
      from_port    = 443
      ipv6_cidr_blocks = []
      prefix_list_ids = []
      protocol     = "tcp"
      security_groups = []
      self         = false
      to_port      = 443
    },
    {
      cidr_blocks = [
        "0.0.0.0/0",
      ]
      from_port    = 80
      ipv6_cidr_blocks = []
      prefix_list_ids = []
      protocol     = "tcp"
      security_groups = []
      self         = false
      to_port      = 80
    },
  ]
  egress            = [
    {
      cidr_blocks = [
        "0.0.0.0/0",
      ]
      from_port    = 0
      ipv6_cidr_blocks = []
      prefix_list_ids = []
      protocol     = "tcp"
      security_groups = []
      self         = false
      to_port      = 80
    },
  ]
}

```

2. Create backend security group with appropriate rules

```

# module.aws_security_group.backend_sg will be created
resource "aws_security_group" "backend_sg" {
  name               = "backend_sg"
  description        = "Security group for backend servers"
  vpc_id             = var.vpc_id
  ingress            = [
    {
      cidr_blocks = [
        "0.0.0.0/0",
      ]
      from_port    = 0
      ipv6_cidr_blocks = []
      prefix_list_ids = []
      protocol     = "tcp"
      security_groups = []
      self         = false
      to_port      = 80
    },
  ]
  egress            = [
    {
      cidr_blocks = [
        "0.0.0.0/0",
      ]
      from_port    = 0
      ipv6_cidr_blocks = []
      prefix_list_ids = []
      protocol     = "tcp"
      security_groups = []
      self         = false
      to_port      = 80
    },
  ]
}

```

3. Use security group IDs for backend ingress (not CIDR blocks)

4. Add descriptive names and tags

main.tf:

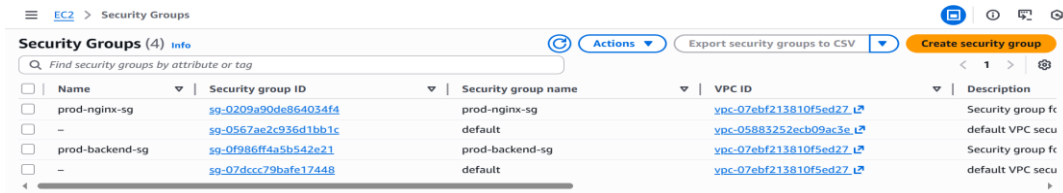
```

# module.aws_security_group.nginx_sg will be created
resource "aws_security_group" "nginx_sg" {
  name               = "${var.env_prefix}-nginx-sg"
  description        = "Security group for Nginx"
  vpc_id             = var.vpc_id
  ingress            = [
    {
      from_port      = 22
      to_port        = 22
      protocol       = "tcp"
      cidr_blocks    = [var.my_ip]
    },
    {
      from_port      = 80
      to_port        = 80
      protocol       = "tcp"
      cidr_blocks    = ["0.0.0.0/0"]
    },
    {
      from_port      = 443
      to_port        = 443
      protocol       = "tcp"
      cidr_blocks    = ["0.0.0.0/0"]
    },
  ]
  egress            = [
    {
      from_port      = 0
      to_port        = 0
      protocol       = "tcp"
      cidr_blocks    = ["0.0.0.0/0"]
    },
  ]
  tags = {
    Name = "${var.env_prefix}-nginx-sg"
  }
}

resource "aws_security_group" "backend_sg" {
  name               = "${var.env_prefix}-backend-sg"
  description        = "Security group for backend servers"
  vpc_id             = var.vpc_id
  ingress            = [
    {
      from_port      = 80
      to_port        = 80
      protocol       = "tcp"
      cidr_blocks    = ["0.0.0.0/0"]
    },
  ]
  egress            = [
    {
      from_port      = 0
      to_port        = 0
      protocol       = "tcp"
      cidr_blocks    = ["0.0.0.0/0"]
    },
  ]
  tags = {
    Name = "${var.env_prefix}-backend-sg"
  }
}

```


AWS Console:



☐	Name	Security group ID	Security group name	VPC ID	Description
☐	prod-nginx-sg	sg-0209a90de864034f4	prod-nginx-sg	vpc-07ebf213810f5ed27	Security group for prod-nginx-sg
☐	-	sg-0567ae2c936d1bb1c	default	vpc-05883252ecb09ac3e	default VPC security group
☐	prod-backend-sg	sg-0f986ff4a5b542e21	prod-backend-sg	vpc-07ebf213810f5ed27	Security group for prod-backend-sg
☐	-	sg-07dccc79bafae17448	default	vpc-07ebf213810f5ed27	default VPC security group

1.4 Locals configuration

TASK:

1. Implement dynamic IP detection
2. Define common tags
3. Define backend server list
4. Add any other reusable local values

locals.tf:

```
PS D:\F3M\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> more locals.tf
# ----- Dynamic IP Detection -----
data "http" "my_ip" {
  url = "https://icanhazip.com"
}

# ----- Local Values -----
locals {
  # Your public IP (for security group rules, etc.)
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"

  # Common tags for all resources
  common_tags = {
    Environment = var.env_prefix
    Project     = "Assignment-2"
    ManagedBy   = "Terraform"
  }

  # Resource naming convention
  name_prefix = "${var.env_prefix}-assignment2"

  # Backend server configurations
  backend_servers = [
    {
      name       = "web-1"
      suffix     = "1"
      script_path = "./scripts/apache-setup.sh"
    },
    {
      name       = "web-2"
      suffix     = "2"
      script_path = "./scripts/nginx-setup.sh"
    }
  ]
}
```

Part 2: Webserver module

2.1 Module design (webserver module)

TASK:

1. Create variables.tf with all required variables
2. Create main.tf with key pair and instance resources
3. Create outputs.tf with all required outputs
4. Ensure module is reusable for both Nginx and backend servers

Variables.tf:

```
PS D:\F3M\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2\modules\webserver> more variables.tf
variable "env_prefix" {
  description = "Environment prefix (e.g. dev, prod)"
  type       = string
}

variable "instance_name" {
  description = "Base name of the EC2 instance"
  type       = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type       = string
}

variable "availability_zone" {
  description = "Availability zone for EC2 instance"
  type       = string
}

variable "vpc_id" {
  description = "VPC ID where instance will be deployed"
  type       = string
}

variable "subnet_id" {
  description = "Subnet ID for EC2 instance"
  type       = string
}

variable "security_group_id" {
  description = "Security group ID for EC2 instance"
  type       = string
}

variable "public_key" {
  description = "Path to public SSH key"
  type       = string
}
```

main.tf:

```
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2\modules\webserver> more main.tf
# ----- AMI Lookup -----
data "aws_ami" "amazon_linux_2023" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

# ----- Key Pair -----
resource "aws_key_pair" "this" {
  key_name   = "${var.env_prefix}-${var.instance_name}-${var.instance_suffix}-key"
  public_key = file(var.public_key)

  tags = merge(
    var.common_tags,
    {
      Name = "${var.instance_name}-key"
    }
  )
}

# ----- EC2 Instance -----
resource "aws_instance" "this" {
  ami              = data.aws_ami.amazon_linux_2023.id
  instance_type    = var.instance_type
  availability_zone = var.availability_zone
  subnet_id        = var.subnet_id
  vpc_security_group_ids = [var.security_group_id]
  associate_public_ip_address = true
  key_name         = aws_key_pair.this.key_name

  user_data = file(var.script_path)

  tags = merge(
    var.common_tags,
    {
      Name = "${var.env_prefix}-${var.instance_name}-${var.instance_suffix}"
    }
  )
}
```

Outputs.tf:

```
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2\modules\webserver> more outputs.tf
output "instance_id" {
  description = "EC2 instance ID"
  value       = aws_instance.this.id
}

output "public_ip" {
  description = "Public IP of EC2 instance"
  value       = aws_instance.this.public_ip
}

output "private_ip" {
  description = "Private IP of EC2 instance"
  value       = aws_instance.this.private_ip
}
```

2.2 Module usage (instantiation)

TASK:

1. Create Nginx server module instance
2. Create backend server module instances using for_each
3. Ensure proper security group assignment
4. Pass all required variables

main.tf:

```
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> more main.tf
module "networking" {
  source = "../modules/networking"
  vpc_cidr_block = var.vpc_cidr_block
  subnet_cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  env_prefix = var.env_prefix
}

module "security" {
  source = "../modules/security"
  vpc_id = module.networking.vpc_id
  env_prefix = var.env_prefix
  my_ip = local.my_ip
}

# ----- Nginx Server -----
module "nginx_server" {
  source = "../modules/webserver"

  env_prefix = var.env_prefix
  instance_name = "nginx-proxy"
  instance_type = var.instance_type
  availability_zone = var.availability_zone
  vpc_id = module.networking.vpc_id
  subnet_id = module.networking.subnet_id
  security_group_id = module.security.nginx_sg_id
  public_key = var.public_key
  script_path = "../scripts/nginx-setup.sh"
  instance_suffix = "nginx"
  common_tags = local.common_tags
}

# ----- Backend Servers -----
module "backend_servers" {
  for_each = {
    for server in local.backend_servers :
      server.name => server
  }

  source = "../modules/webserver"

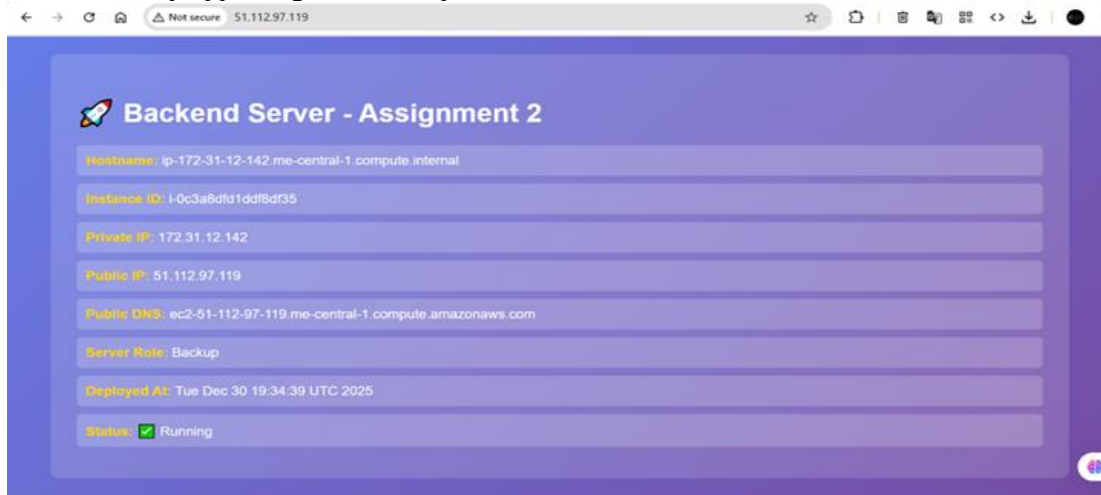
  env_prefix = var.env_prefix
  instance_name = each.value.name
  More --
}
```

Part 3: Server configuration scripts

3.1 Apache backend script (apache-setup.sh)

TASK:

1. Create the script with all required features
2. Ensure it uses IMDSv2 for metadata
3. Create visually appealing HTML output



4. Make script executable

```
[ec2-user@ip-172-31-12-142 ~]$ chmod +x apache-setup.sh
```

5. Test script independently

```
[ec2-user@ip-172-31-12-142 ~]$ sudo ./apache-setup.sh

Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Nothing to do.
Complete!
Last metadata expiration check: 0:00:02 ago on Tue Dec 30 19:34:34 2025.
Dependencies resolved.

=====
Package                               Architecture      Version            Repository         Size
=====
Installing:
httpd                                  x86_64            2.4.65-1.amzn2023.0.2    amazonlinux        47 k
Installing dependencies:
apr-1.7.5-1.amzn2023.0.4.x86_64.rpm  x86_64            1.7.5-1.amzn2023.0.4    amazonlinux        129 k
apr-util-1.6.3-1.amzn2023.0.2.x86_64 rpm  x86_64            1.6.3-1.amzn2023.0.2    amazonlinux        97 k
apr-util-1.6.3-1.amzn2023.0.2.x86_64 rpm  x86_64            1.6.3-1.amzn2023.0.2    amazonlinux        13 k
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch  x86_64            2.4.65-1.amzn2023.0.3    amazonlinux        1.4 M
httpd-core-2.4.65-1.amzn2023.0.2.x86_64 rpm  x86_64            2.4.65-1.amzn2023.0.2    amazonlinux        13 k
httpd-tools-2.4.65-1.amzn2023.0.2.x86_64 rpm  x86_64            2.4.65-1.amzn2023.0.2    amazonlinux        81 k
libbrotli-1.0.9-4.amzn2023.0.2.x86_64 rpm  x86_64            1.0.9-4.amzn2023.0.2    amazonlinux        315 k
mailcap-2.1.49-3.amzn2023.0.3.noarch      x86_64            2.1.49-3.amzn2023.0.3    amazonlinux        13 k
Installing weak dependencies:
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64 rpm  x86_64            1.6.3-1.amzn2023.0.2    amazonlinux        15 k
mod_http2-2.0.27-1.amzn2023.0.3.x86_64 rpm  x86_64            2.0.27-1.amzn2023.0.3    amazonlinux        166 k
mod_lua-2.4.65-1.amzn2023.0.2.x86_64 rpm  x86_64            2.4.65-1.amzn2023.0.2    amazonlinux        68 k
=====
Transaction Summary
Install 13 Packages
Total download size: 2.4 M
Installed size: 6.9 M
Downloading Packages:
(1/13): apr-1.7.5-1.amzn2023.0.4.x86_64.rpm 1.8 MB/s | 97 kB 00:00
(2/13): apr-util-1.6.3-1.amzn2023.0.2.x86_64 rpm 2.2 MB/s | 129 kB 00:00
(3/13): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch 169 kB/s | 13 kB 00:00
(4/13): httpd-core-2.4.65-1.amzn2023.0.2.x86_64 rpm 673 kB/s | 10 kB 00:00
(5/13): httpd-tools-2.4.65-1.amzn2023.0.2.x86_64 rpm 339 kB/s | 15 kB 00:00
(6/13): libbrotli-1.0.9-4.amzn2023.0.2.x86_64 rpm 1.3 MB/s | 47 kB 00:00
(7/13): mailcap-2.1.49-3.amzn2023.0.3.noarch 379 kB/s | 13 kB 00:00
(8/13): mod_http2-2.0.27-1.amzn2023.0.3.x86_64 rpm 2.1 MB/s | 81 kB 00:00
(9/13): mod_lua-2.4.65-1.amzn2023.0.2.x86_64 rpm 10 MB/s | 315 kB 00:00
(10/13): apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64 rpm 15 MB/s | 1.4 MB 00:00
(11/13): httpd-2.4.65-1.amzn2023.0.2.x86_64 rpm 1.1 MB/s | 33 kB 00:00
(12/13): httpd-2.4.65-1.amzn2023.0.2.x86_64 rpm 1.6 MB/s | 68 kB 00:00
(13/13): httpd-2.4.65-1.amzn2023.0.2.x86_64 rpm 3.0 MB/s | 166 kB 00:00
Total 9.0 MB/s | 2.4 MB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction:
Preparing : apr-1.7.5-1.amzn2023.0.4.x86_64 1/13
Installing : apr-1.7.5-1.amzn2023.0.4.x86_64 1/13
Installing : apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64 2/13
Installing : apr-util-1.6.3-1.amzn2023.0.2.x86_64 3/13
Installing : apr-util-1.6.3-1.amzn2023.0.2.x86_64 4/13
Installing : mailcap-2.1.49-3.amzn2023.0.3.noarch 5/13
Installing : httpd-tools-2.4.65-1.amzn2023.0.2.x86_64 6/13
Installing : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 7/13
Running scriptlet: httpd-core-2.4.65-1.amzn2023.0.2.x86_64 8/13
Installing : httpd-core-2.4.65-1.amzn2023.0.2.x86_64 9/13
Installing : httpd-2.4.65-1.amzn2023.0.2.x86_64 10/13
Installing : mod_http2-2.0.27-1.amzn2023.0.3.x86_64 11/13
Installing : mod_lua-2.4.65-1.amzn2023.0.2.x86_64 12/13
Installing : httpd-2.4.65-1.amzn2023.0.2.x86_64 13/13
Running scriptlet: httpd-2.4.65-1.amzn2023.0.2.x86_64 13/13
Verifying : apr-1.7.5-1.amzn2023.0.4.x86_64 1/13
Verifying : apr-util-1.6.3-1.amzn2023.0.2.x86_64 2/13
Verifying : apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64 3/13
Verifying : httpd-core-2.4.65-1.amzn2023.0.2.x86_64 4/13
Verifying : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch 5/13
Verifying : httpd-2.4.65-1.amzn2023.0.2.x86_64 6/13
Verifying : httpd-core-2.4.65-1.amzn2023.0.2.x86_64 7/13
Verifying : httpd-tools-2.4.65-1.amzn2023.0.2.x86_64 8/13
Verifying : libbrotli-1.0.9-4.amzn2023.0.2.x86_64 9/13
Verifying : mailcap-2.1.49-3.amzn2023.0.3.noarch 10/13
Verifying : mod_http2-2.0.27-1.amzn2023.0.3.x86_64 11/13
Verifying : mod_lua-2.4.65-1.amzn2023.0.2.x86_64 12/13
Verifying : httpd-2.4.65-1.amzn2023.0.2.x86_64 13/13
Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64 apr-util-1.6.3-1.amzn2023.0.2.x86_64 apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64 httpd-2.4.65-1.amzn2023.0.2.x86_64
httpd-core-2.4.65-1.amzn2023.0.2.x86_64 httpd-tools-2.4.65-1.amzn2023.0.2.x86_64 mailcap-2.1.49-3.amzn2023.0.3.noarch mod_lua-2.4.65-1.amzn2023.0.2.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
Complete!
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
```

3.2 Nginx server setup script (nginx-setup.sh)

TASK:

1. Create script with SSL certificate generation
2. Configure upstream with placeholder IPs
3. Implement caching
4. Add security headers
5. Configure HTTP to HTTPS redirect
6. Add health check endpoint

nginx-setup.sh:

```
[ec2-user@ip-172-31-12-142 ~]$ sudo ./nginx-setup.sh
Last metadata expiration check: 0:52:15 ago on Tue Dec 30 19:34:34 2025.
Dependencies resolved.
Nothing to do.
Complete!
Last metadata expiration check: 0:52:16 ago on Tue Dec 30 19:34:34 2025.
Package openssl-1:3.2.2-1.amzn2023.0.2.x86_64 is already installed.
Dependencies resolved.

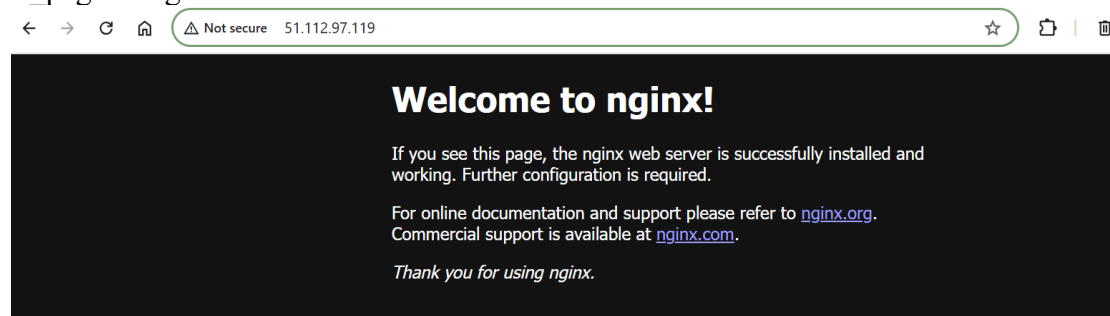
=====
Package                Architecture      Version              Repository           Size
=====
Installing:
nginx                  x86_64            1:1.28.0-1.amzn2023.0.2    amazonlinux           33 k
Installing dependencies:
gperf-libs-2.9.1-1.amzn2023.0.3.x86_64 rpm      2.9.1-1.amzn2023.0.3    amazonlinux           308 k
libumwind-1.4.0-5.amzn2023.0.3.x86_64 rpm      1.4.0-5.amzn2023.0.3    amazonlinux           66 k
nginx-core-1:1.28.0-1.amzn2023.0.2.x86_64 rpm    1:1.28.0-1.amzn2023.0.2    amazonlinux           686 k
nginx-filesystem-1:1.28.0-1.amzn2023.0.2.noarch rpm  1:1.28.0-1.amzn2023.0.2    amazonlinux           9.6 k
nginx-minetypes-2.1.49-3.amzn2023.0.3.noarch rpm    2.1.49-3.amzn2023.0.3    amazonlinux           21 k
Transaction Summary
-----
Install 6 Packages
Total download size: 1.1 M
Installed size: 3.6 M
Downloading Packages:
(1/6): nginx-1.28.0-1.amzn2023.0.2.x86_64.rpm      756 kb/s | 33 kb  00:00
(2/6): libumwind-1.4.0-5.amzn2023.0.3.x86_64.rpm    1.4 MB/s | 66 kb  00:00
(3/6): gperf-libs-2.9.1-1.amzn2023.0.3.x86_64.rpm    5.2 MB/s | 308 kb  00:00
(4/6): nginx-filesystem-1.28.0-1.amzn2023.0.2.noarch.rpm  350 kb/s | 9.6 kb  00:00
(5/6): nginx-minetypes-2.1.49-3.amzn2023.0.3.noarch.rpm  839 kb/s | 21 kb  00:00
(6/6): nginx-core-1.28.0-1.amzn2023.0.2.x86_64.rpm    14 MB/s | 686 kb  00:00
-----
Total: 8.5 MB/s | 1.1 MB  00:00

Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing : nginx-filesystem-1:1.28.0-1.amzn2023.0.2.noarch      1/1
Running scriptlet: nginx-filesystem-1:1.28.0-1.amzn2023.0.2.noarch 1/6
Installing : nginx-minetypes-2.1.49-3.amzn2023.0.3.noarch      1/6
Installing : libumwind-1.4.0-5.amzn2023.0.3.x86_64             3/6
Installing : gperf-libs-2.9.1-1.amzn2023.0.3.x86_64           4/6
Installing : nginx-core-1:1.28.0-1.amzn2023.0.2.x86_64         5/6
Installing : nginx-1:1.28.0-1.amzn2023.0.2.x86_64             6/6
Running scriptlet: nginx-1:1.28.0-1.amzn2023.0.2.x86_64         6/6
Verifying : gperf-libs-2.9.1-1.amzn2023.0.3.x86_64            1/6
Verifying : libumwind-1.4.0-5.amzn2023.0.3.x86_64             2/6
Verifying : nginx-1:1.28.0-1.amzn2023.0.2.x86_64             3/6
Verifying : nginx-core-1:1.28.0-1.amzn2023.0.2.x86_64         4/6
Verifying : nginx-filesystem-1:1.28.0-1.amzn2023.0.2.noarch    5/6
Verifying : nginx-minetypes-2.1.49-3.amzn2023.0.3.noarch      6/6

Installed:
gperf-libs-2.9.1-1.amzn2023.0.3.x86_64    libumwind-1.4.0-5.amzn2023.0.3.x86_64    nginx-1:1.28.0-1.amzn2023.0.2.x86_64    nginx-core-1:1.28.0-1.amzn2023.0.2.x86_64
nginx-filesystem-1:1.28.0-1.amzn2023.0.2.noarch  nginx-minetypes-2.1.49-3.amzn2023.0.3.noarch

Complete!
```

default_page of nginx:



Part 4: Infrastructure Deployment

4.1 Initial Deployment (SSH keys, terraform init/plan/apply)

TASK:

1. Generate SSH key pair if not exists

☐ assignment2-key rsa 2025/12/30 23:32 GMT+5 b0:f5:6a:52:74:84:39:b7:3... key-0c8ea4a47ad1f7275

2. Initialize Terraform (terraform init)

```
PS D:\F\JW\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> terraform init
Initializing the backend...
Initializing modules...
Initializing provider plugins...
- Reusing previous version of hashicorp/http from the dependency lock file
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously installed hashicorp/http v3.5.0
- Using previously installed hashicorp/aws v6.27.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you want to change modules or backend configuration for Terraform,
run this command to reinitialize your working directory. If you forget, other
commands will output a warning to remind you to do so at the next step.
```

3. Validate configuration (terraform validate)

```
PS D:\F3WU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> terraform validate
Success! The configuration is valid.
```

4. Plan deployment (terraform plan)

```
PS D:\F3WU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> terraform plan
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 1s [id=https://icanhazip.com]
module.backend_servers["web-1"].data.aws_ami.amazon_linux_2023: Reading...
module.nginx_server.data.aws_ami.amazon_linux_2023: Reading...
module.backend_servers["web-2"].data.aws_ami.amazon_linux_2023: Reading...
module.nginx_server.aws_key_pair.this: Refreshing state... [id=prod-web-2-1-key]
module.backend_servers["web-1"].aws_key_pair.this: Refreshing state... [id=prod-web-1-1-key]
module.backend_servers["web-1"].aws_key_pair.this: Refreshing state... [id=prod-web-1-2-key]
module.backend_servers["web-2"].data.aws_ami.amazon_linux_2023: Read complete after 1s [id=ami-009ce8169fc88edf5]
module.backend_servers["web-2"].data.aws_ami.amazon_linux_2023: Read complete after 1s [id=ami-009ce8169fc88edf5]
module.backend_servers["web-1"].data.aws_ami.amazon_linux_2023: Read complete after 1s [id=ami-009ce8169fc88edf5]
module.backend_servers["web-1"].data.aws_ami.amazon_linux_2023: Read complete after 1s [id=ami-009ce8169fc88edf5]
module.networking.aws_internet_gateway.this: Refreshing state... [id=igw-8d0c8e96251d6ac51]
module.networking.aws_subnet.this: Refreshing state... [id=subnet-8c308eb0839f5a21]
module.security.aws_security_group.nginx_sg: Refreshing state... [id=sg-8009b08a686a0464]
module.networking.aws_route_table.this: Refreshing state... [id=rtb-0fd41cb2165f64de]
module.security.aws_security_group.backend_sg: Refreshing state... [id=sg-0f906f44d5042e21]
module.nginx_server.aws_instance.this: Refreshing state... [id=i-03a5177c480dc160]
module.networking.aws_route_table_association.this: Refreshing state... [id=rtbassoc-0a5c37c71a5be499f]
module.backend_servers["web-2"].aws_instance.this: Refreshing state... [id=i-0a47b0ff5cfd7f5]
module.backend_servers["web-1"].aws_instance.this: Refreshing state... [id=i-00f7978fd22180e78]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:
```

```
# module.backend_servers["web-2"].aws_instance.this will be updated in-place
~ resource "aws_instance" "this" {
  id              = "i-0a47b0ff5cfd7f5"
  public_dns      = (known after apply)
  public_ip       = "3.28.137.183" -> (known after apply)
  tags            = {
    "Environment" = "prod"
    "ManagedBy"  = "Terraform"
    "Name"         = "prod-web-2-2"
    "Project"      = "Assignment-2"
  }
  user_data       = <<-EOF
#!/bin/bash
set -e

# Update and install Nginx
yum update -y
yum install -y nginx openssl
systemctl start nginx
systemctl enable nginx

# Create SSL directories
mkdir -p /etc/ssl/private
mkdir -p /etc/ssl/certs

# Get metadata token
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# Get public IP
PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-ip)

# Generate self-signed certificate
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/selfsigned.key \
-out /etc/ssl/certs/selfsigned.crt \
-subj "/CN=$PUBLIC_IP" \
```

5. Apply configuration (terraform apply -auto-approve)

```
PS D:\F3WU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> terraform apply -auto-approve
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
module.backend_servers["web-1"].data.aws_ami.amazon_linux_2023: Reading...
module.backend_servers["web-2"].data.aws_ami.amazon_linux_2023: Reading...
module.nginx_server.data.aws_ami.amazon_linux_2023: Reading...
module.backend_servers["web-1"].aws_key_pair.this: Refreshing state... [id=prod-web-2-1-key]
module.backend_servers["web-2"].aws_key_pair.this: Refreshing state... [id=prod-web-1-1-key]
module.backend_servers["web-1"].aws_key_pair.this: Refreshing state... [id=prod-web-1-2-key]
module.backend_servers["web-2"].data.aws_ami.amazon_linux_2023: Read complete after 1s [id=ami-009ce8169fc88edf5]
module.backend_servers["web-1"].data.aws_ami.amazon_linux_2023: Read complete after 1s [id=ami-009ce8169fc88edf5]
module.backend_servers["web-2"].data.aws_ami.amazon_linux_2023: Read complete after 1s [id=ami-009ce8169fc88edf5]
module.backend_servers["web-1"].data.aws_ami.amazon_linux_2023: Read complete after 1s [id=ami-009ce8169fc88edf5]
module.networking.aws_internet_gateway.this: Refreshing state... [id=igw-8d0c8e96251d6ac51]
module.networking.aws_subnet.this: Refreshing state... [id=subnet-8c308eb0839f5a21]
module.security.aws_security_group.nginx_sg: Refreshing state... [id=sg-8009b08a686a0464]
module.networking.aws_route_table.this: Refreshing state... [id=rtb-0fd41cb2165f64de]
module.security.aws_security_group.backend_sg: Refreshing state... [id=sg-0f906f44d5042e21]
module.nginx_server.aws_instance.this: Refreshing state... [id=i-03a5177c480dc160]
module.networking.aws_route_table_association.this: Refreshing state... [id=rtbassoc-0a5c37c71a5be499f]
module.backend_servers["web-2"].aws_instance.this: Refreshing state... [id=i-0a47b0ff5cfd7f5]
module.backend_servers["web-1"].aws_instance.this: Refreshing state... [id=i-00f7978fd22180e78]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ update in-place

Terraform will perform the following actions:
```

```
# module.backend_servers["web-2"].aws_instance.this will be updated in-place
~ resource "aws_instance" "this" {
  id              = "i-0a47b0ff5cfd7f5"
  public_dns      = (known after apply)
  public_ip       = "3.28.137.183" -> (known after apply)
  tags            = {
    "Environment" = "prod"
    "ManagedBy"  = "Terraform"
    "Name"         = "prod-web-2-2"
    "Project"      = "Assignment-2"
  }
  user_data       = <<-EOF
#!/bin/bash
set -e

# Update and install Nginx
yum update -y
yum install -y nginx openssl
systemctl start nginx
systemctl enable nginx

# Create SSL directories
mkdir -p /etc/ssl/private
mkdir -p /etc/ssl/certs

# Get metadata token
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# Get public IP
PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-ip)

# Generate self-signed certificate
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/selfsigned.key \
-out /etc/ssl/certs/selfsigned.crt \
-subj "/CN=$PUBLIC_IP" \
-addext "subjectAltName=IP:$PUBLIC_IP" \
-addext "basicConstraints=CA:FALSE" \
-addext "keyUsage=digitalSignature,keyEncipherment" \
-addext "extendedKeyUsage=serverAuth" \
```

4.2 Output configuration

TASK:

1. Create all required outputs
2. Add helpful descriptions
3. Include configuration guide
4. Display outputs after apply

terraform output:

```
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> terraform output
backend_servers_info = {
  "web-1" = {
    "public_ip" = "40.172.122.79"
  }
  "web-2" = {
    "instance_id" = "i-0ba47bdf5cfd7fb5"
    "private_ip" = "10.0.10.141"
    "public_ip" = "3.28.253.236"
  }
  "web-3" = {
    "instance_id" = "i-06118191409fd8d2"
    "private_ip" = "10.0.10.207"
    "public_ip" = "3.28.205.209"
  }
}
configuration_guide = <<EOT
=====
DEPLOYMENT SUCCESSFUL!
=====
Next Steps:
1. SSH into Nginx server: ssh ec2-user@51.112.52.65
2. Edit Nginx config: sudo vim /etc/nginx/nginx.conf
3. Update backend IPs in upstream block:
  - BACKEND_IP_1: 10.0.10.22
  - BACKEND_IP_2: 10.0.10.141
  - BACKEND_IP_3: 10.0.10.207
4. Restart Nginx: sudo systemctl restart nginx
5. Test: https://51.112.52.65

Backend Servers:
- web-1: 40.172.122.79 (private: 10.0.10.22)
- web-2: 3.28.253.236 (private: 10.0.10.141)
- web-3: 3.28.205.209 (private: 10.0.10.207)

=====
EOT
nginx_instance_id = "i-03451f72c489de156"
nginx_public_ip = "51.112.52.65"
subnet_id = "subnet-0c808ebb6863f6542"
vpc_id = "vpc-07ebf213810f5ed27"
```

outputs json:

```
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> terraform output -json > outputs.json
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2>
```

4.3 AWS Console Verification (manual checks)

TASK:

1. Verify VPC created

Your VPCs

VPCs | VPC encryption controls

Your VPCs (2) Info Last updated 2 minutes ago Actions Create VPC

Find VPCs by attribute or tag

☐	Name	VPC ID	State	Encryption c...	Encryption contr...
☐	prod-vpc	vpc-07ebf213810f5ed27	Available	-	-
☐	-	vpc-05883252ecb09ac3e	Available	-	-

2. Verify Subnet created

Subnets (4) Info Last updated 5 minutes ago Actions Create subnet

Find subnets by attribute or tag

☐	Name	Subnet ID	State	VPC	Bloc...	IPv4 CIDR
☐	prod-subnet	subnet-0c808ebb6863f6542	Available..	vpc-07ebf213810f5ed27 p...	Off	10.0.10.0/24
☐	-	subnet-015aa793c89df0a82	Available..	vpc-05883252ecb09ac3e	Off	172.31.32.0/20
☐	-	subnet-084c39d400b49a9b4	Available..	vpc-05883252ecb09ac3e	Off	172.31.16.0/20
☐	-	subnet-0336f1ba99f13aac0	Available..	vpc-05883252ecb09ac3e	Off	172.31.0.0/20

3. Verify Internet Gateway attached

Internet gateways (2) Info Last updated 9 minutes ago Actions Create internet gateway

Find internet gateways by attribute or tag

☐	Name	Internet gateway ID	State	VPC ID	Owner
☐	-	igw-056b5f9496be95def	Attached	vpc-05883252ecb09ac3e	941618064465
☐	prod-igw	igw-0dec8e96251d6ac51	Attached	vpc-07ebf213810f5ed27 prod-vpc	941618064465

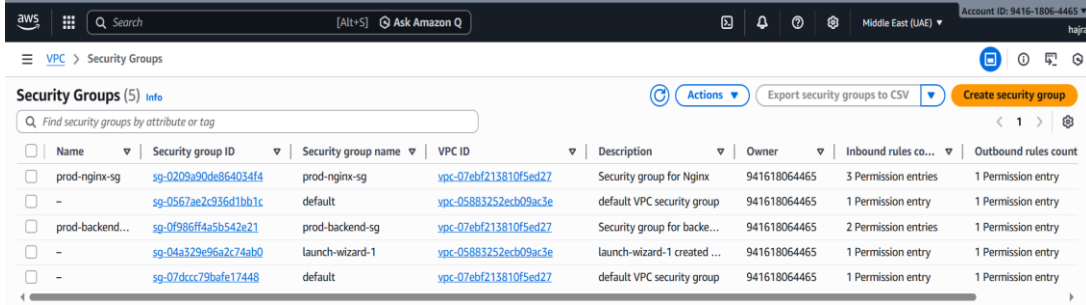
4. Verify Route Table configured

Route tables (3) Info Last updated 9 minutes ago Actions Create route table

Find route tables by attribute or tag

☐	Name	Route table ID	Explicit subnet ass...	Edge...	Main	VPC	Own...
☐	-	rtb-063f5be50e6c82c28	-	-	Yes	vpc-07ebf213810f5...	941618...
☐	prod-rt	rtb-0fdf41cb2165f648e	subnet-0c808ebb686...	-	No	vpc-07ebf213810f5...	941618...
☐	-	rtb-025c11b360da41841	-	-	Yes	vpc-05883252ecb0...	941618...

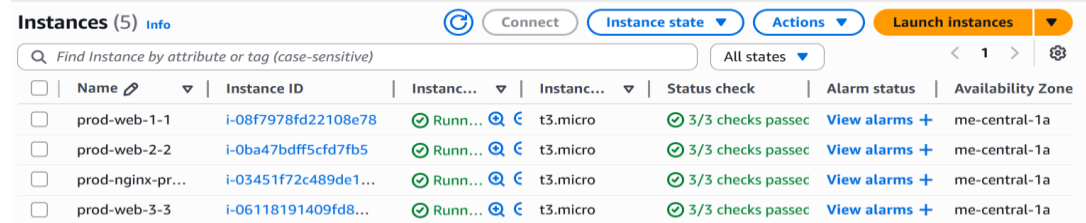
5. Verify Security Groups created with correct rules



The screenshot shows the AWS Management Console for Security Groups. It lists five security groups with their respective IDs, names, VPC IDs, descriptions, owners, and rule counts.

Name	Security group ID	Security group name	VPC ID	Description	Owner	Inbound rules count	Outbound rules count
prod-nginx-sg	sg-0209a90de864034f4	prod-nginx-sg	vpc-07ebf213810f5ed27	Security group for Nginx	941618064465	3 Permission entries	1 Permission entry
-	sg-0567ae2c336d1bb1c	default	vpc-05883252eb09ac3e	default VPC security group	941618064465	1 Permission entry	1 Permission entry
prod-backend...	sg-0f986ff4a5b542e21	prod-backend-sg	vpc-07ebf213810f5ed27	Security group for backe...	941618064465	2 Permission entries	1 Permission entry
-	sg-04a329e96a2c74ab0	launch-wizard-1	vpc-05883252eb09ac3e	launch-wizard-1 created ...	941618064465	1 Permission entry	1 Permission entry
-	sg-07dccc79baf617448	default	vpc-07ebf213810f5ed27	default VPC security group	941618064465	1 Permission entry	1 Permission entry

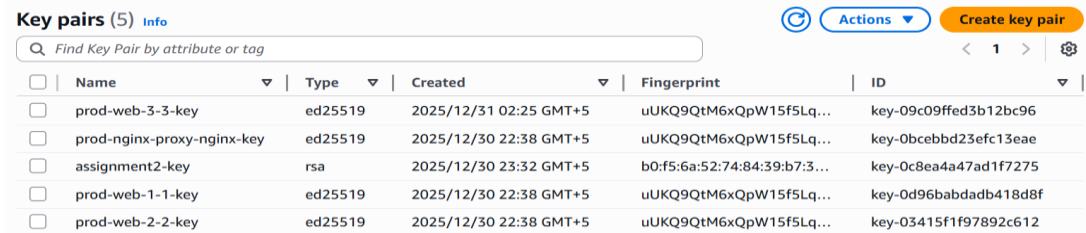
6. Verify all 4 EC2 instances running



The screenshot shows the AWS Management Console for EC2 Instances. It lists five instances, all of which are in the 'Running' state.

Name	Instance ID	Instanc...	Instanc...	Status check	Alarm status	Availability Zone
prod-web-1-1	i-08f7978fd22108e78	Runn...	t3.micro	3/3 checks pass	View alarms +	me-central-1a
prod-web-2-2	i-0ba47bdf5cf7fb5	Runn...	t3.micro	3/3 checks pass	View alarms +	me-central-1a
prod-nginx-pr...	i-03451f72c489de1...	Runn...	t3.micro	3/3 checks pass	View alarms +	me-central-1a
prod-web-3-3	i-06118191409fd8...	Runn...	t3.micro	3/3 checks pass	View alarms +	me-central-1a

7. Verify Key Pairs created



The screenshot shows the AWS Management Console for Key Pairs. It lists five key pairs, all of which are in the 'Active' state.

Name	Type	Created	Fingerprint	ID
prod-web-3-3-key	ed25519	2025/12/31 02:25 GMT+5	uUKQ9QtM6xQpW15f5Lq...	key-09c09ffed3b12bc96
prod-nginx-proxy-nginx-key	ed25519	2025/12/30 22:38 GMT+5	uUKQ9QtM6xQpW15f5Lq...	key-0bcebbd23efc13eae
assignment2-key	rsa	2025/12/30 23:32 GMT+5	b0:f5:6a:52:74:84:39:b7:3...	key-0c8ea4a47ad1f7275
prod-web-1-1-key	ed25519	2025/12/30 22:38 GMT+5	uUKQ9QtM6xQpW15f5Lq...	key-0d96babdbadb418d8f
prod-web-2-2-key	ed25519	2025/12/30 22:38 GMT+5	uUKQ9QtM6xQpW15f5Lq...	key-03415f1f97892c612

Part 5: Nginx Configuration & Testing

5.1 Update Nginx backend configuration

TASK:

1. SSH into Nginx server

```
PS D:\F3WU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> ssh -i "keys/assignment2-key.pem" ec2-user@3.28.219.37
The authenticity of host '3.28.219.37 (3.28.219.37)' can't be established.
ED25519 key fingerprint is SHA256:R/+wyfHCqx2wrFsvdKZnOh10qbxwhvoSVVwMCEnsy6Y.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.28.219.37' (ED25519) to the list of known hosts.

~
#####
      \#####\
       \###|
        \#/
         V~.'->
          /
         / \
        /   \
       /     \
      /       \
     /         \
    /           \
   /             \
  /               \
 /                 \
/                   \
[ec2-user@ip-172-31-35-93 ~]$
```

2. Edit /etc/nginx/nginx.conf

```
[ec2-user@ip-172-31-35-93 ~]$ sudo vim /etc/nginx/nginx.conf
```

3. Replace placeholder IPs with actual private IPs of backend servers

```
root@ip-172-31-35-93:~# cat /etc/nginx/nginx.conf
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    upstream backend_servers {
        server 10.0.10.22:80; # prod-web-1-1
        server 10.0.10.141:80; # prod-web-2-2
        server 10.0.10.207:80 backup; # prod-web-3-3
    }

    server {
        listen 80;

        location / {
            proxy_pass http://backend_servers;
        }
    }
}
```

4. Test Nginx configuration

```
[root@ip-172-31-35-93 ~]# sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

5. Restart Nginx service

```
[root@ip-172-31-35-93 ~]# sudo systemctl restart nginx
[root@ip-172-31-35-93 ~]# sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
   Active: active (running) since Wed 2025-12-31 09:47:45 UTC; 14s ago
     Process: 28232 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 28235 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 28240 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
    Main PID: 28248 (nginx)
      Tasks: 3 (limit: 1067)
     Memory: 2.9M
        CPU: 27ms
    CGroup: /system.slice/nginx.service
            └─28248 "nginx: master process /usr/sbin/nginx"
              └─28249 "nginx: worker process"
                └─28250 "nginx: worker process"

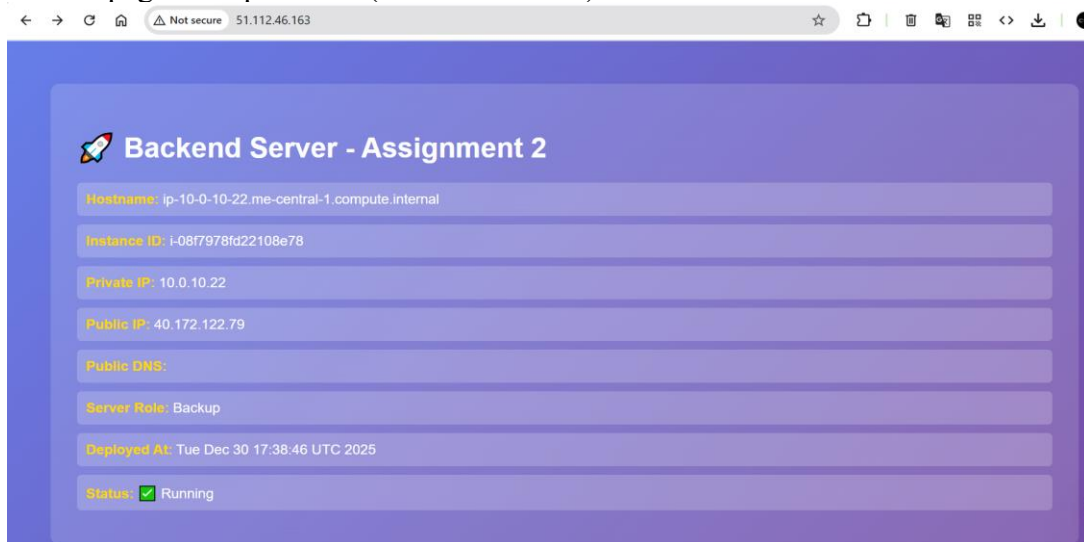
Dec 31 09:47:45 ip-172-31-35-93.me-central-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Dec 31 09:47:45 ip-172-31-35-93.me-central-1.compute.internal nginx[28235]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Dec 31 09:47:45 ip-172-31-35-93.me-central-1.compute.internal nginx[28235]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Dec 31 09:47:45 ip-172-31-35-93.me-central-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
```

5.2 Test load balancing

TASK:

```
[ec2-user@ip-10-0-10-203 ~]$ curl http://51.112.46.163
<!DOCTYPE html>
<html>
<head>
  <title>Backend Web Server</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 50px;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      color: white;
    }
    .container {
      background: rgba(255,255,255,0.15);
      padding: 30px;
      border-radius: 10px;
    }
    .info {
      margin: 10px 0;
      padding: 10px;
      background: rgba(255,255,255,0.2);
      border-radius: 5px;
    }
    .label { font-weight: bold; color: #ffd700; }
  </style>
</head>
<body>
  <div class="container">
    <h1> Backend Server - Assignment 2</h1>
    <div class="info"><span class="label">Hostname:</span> ip-10-0-10-22.me-central-1.compute.internal</div>
    <div class="info"><span class="label">Instance ID:</span> i-08f7978fd22108e78</div>
    <div class="info"><span class="label">Private IP:</span> 10.0.10.22</div>
    <div class="info"><span class="label">Public IP:</span> 40.172.122.79</div>
    <div class="info"><span class="label">Public DNS:</span> </div>
    <div class="info"><span class="label">Server Role:</span> Backup</div>
    <div class="info"><span class="label">Deployed At:</span> Tue Dec 30 17:38:46 UTC 2025</div>
    <div class="info"><span class="label">Status:</span> Running</div>
  </div>
</body>
</html>
```

1. Open browser to https://
2. Accept the security warning for self-signed certificate
3. Reload page multiple times (at least 10 times)



4. Verify traffic alternates between web-1 and web-2

 **Backend Server - Assignment 2**

Hostname: ip-10-0-0-10-141.me-central-1.compute.internal

Instance ID: i-0ba47bdf5cfd7fb5

Private IP: 10.0.10.141

Public IP: 3.28.253.236

Public DNS:

Server Role: Backup

Deployed At: Tue Dec 30 17:38:46 UTC 2025

Status:  Running

5. Verify web-3 is NOT serving traffic (it's backup only)

6. Load balancing pattern:

Nginx is configured as a reverse proxy and load balancer in front of two active web servers (web-1 and web-2) using a round-robin strategy.

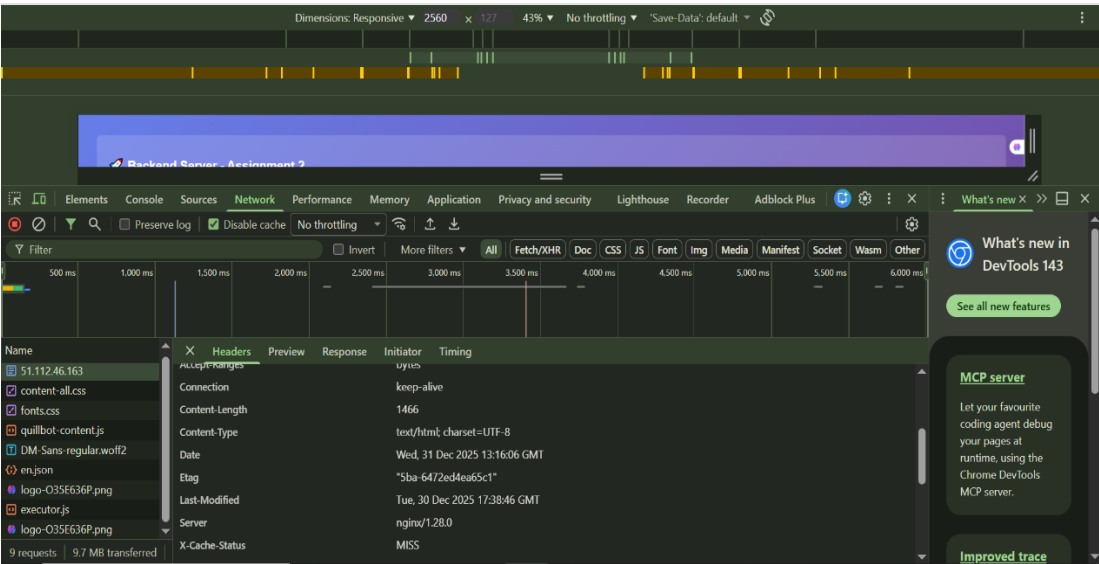
When the page is reloaded multiple times, traffic alternates between web-1 and web-2, confirming proper load distribution.

The backup server (web-3) does not receive traffic under normal conditions, verifying that it is correctly configured as a backup-only server.

5.3 Test cache functionality (X-Cache-Status)

TASK:

1. Open browser developer tools (F12)
2. Navigate to Network tab
3. Clear browser cache Load
4. Check response headers for X-Cache-Status: MISS (first request)



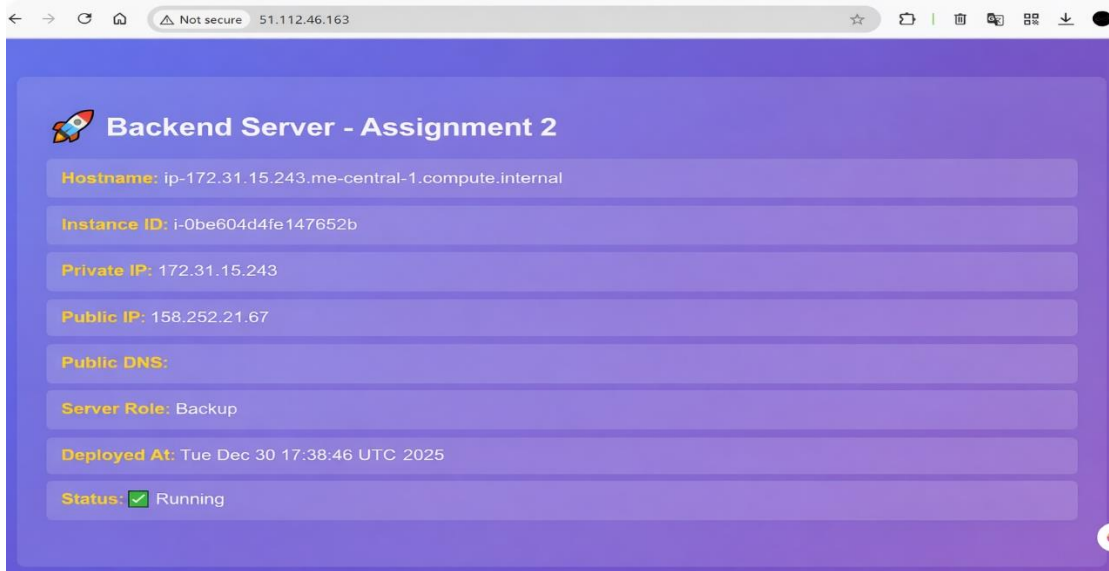
The screenshot shows the Chrome DevTools Network tab. The top panel displays a timeline of network requests. The bottom panel shows the details of a selected request to 51.112.46.163. The 'Headers' tab is active, showing the 'X-Cache-Status' header with a value of 'MISS'. The 'Response' tab shows the HTML content of the page.

Name	Headers	Preview	Response	Initiator	Timing
51.112.46.163	Connection	keep-alive		uyus	
	Content-Length	1466			
	Content-Type	text/html; charset=UTF-8			
	Date	Wed, 31 Dec 2025 13:16:06 GMT			
	Etag	"5ba-6472ed4ea65c1"			
	Last-Modified	Tue, 30 Dec 2025 17:38:46 GMT			
	Server	nginx/1.28.0			
	X-Cache-Status	MISS			

5. Reload page

6. Check response headers for X-Cache-Status: HIT (cached request)

4. Reload Nginx page - should now show web-3 (backup activated)



5. Nginx error log:

```
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> ssh -i "keys/assignment2-key.pem" ec2-user@3.28.42.112
The authenticity of host '3.28.42.112 (3.28.42.112)' can't be established.
ED25519 key fingerprint is SHA256:sg+cw3dtSk65AG0sqRlAGAnuycoBe0y5tnF57Ha6Rk8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.28.42.112' (ED25519) to the list of known hosts.
ec2-user@3.28.42.112: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2>
```

6. Restart web-1 and web-2

7. Verify traffic returns to web-1 and web-2

```
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> ssh -i "keys/assignment2-key.pem" ec2-user@51.112.97.151
~#
#####
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
Last login: Wed Dec 31 16:12:56 2025 from 39.60.152.187
[ec2-user@ip-172-31-0-220 ~]$ sudo systemctl start httpd
[ec2-user@ip-172-31-0-220 ~]$ exit
logout
Connection to 51.112.97.151 closed.
PS D:\FJWU\Semester 5\Cloud Computing\CC Class-Engr Waqas Saleem\Assignment2> ssh -i "keys/assignment2-key.pem" ec2-user@3.28.242.165
~#
#####
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
Last login: Wed Dec 31 16:31:40 2025 from 39.60.152.187
[ec2-user@ip-172-31-9-221 ~]$ sudo systemctl start httpd
```

5.5 Security & performance analysis

TASK:

1. Check SSL/TLS certificate details

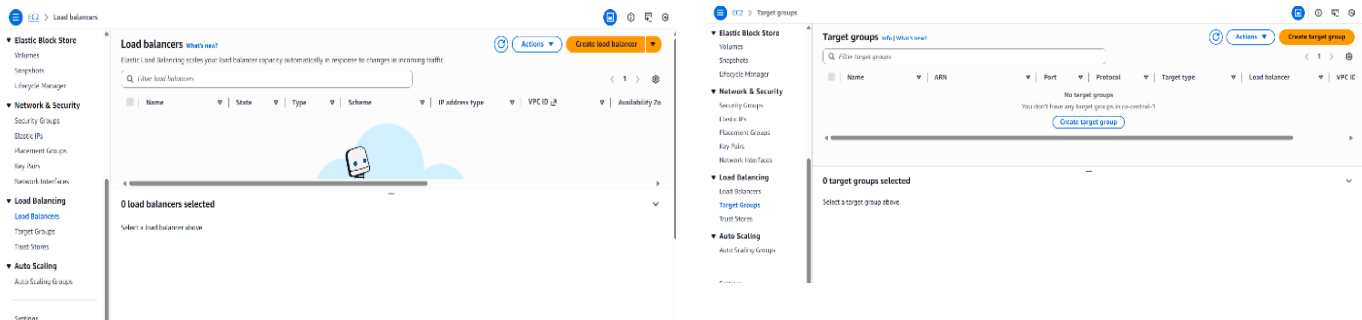
```
Cipher      : TLS_AES_256_GCM_SHA384
Session-ID: B1B04F69F06B52C91A2426B0ECD3E22972595F7B694344F6AF5DC1CCB3DCE64F
Session-ID-ctx:
Resumption PSK: 2B0413154E92E373E60412BF895F77F7C0382DE0A842ADB65CFBE15749D13BE554C9E7EA1641DC6DAE7B77377EF784D
PSK identity: None
PSK identity hint: None
SRP username: None
TLS session ticket lifetime hint: 300 (seconds)
TLS session ticket:
0000 - 49 7c 00 70 80 1d 91 76-02 3a 2f 51 17 34 7e 2e  I\p...v.i/Q.4~.
0010 - 94 19 c8 d4 23 0f 2b 76-fe d2 60 27 63 49 e1 de  ...#o+V...cI..
0020 - af 09 4a 70 fa 3f 0c 52-f2 bb 66 6f bb 95 5c 1a  ...jp.PIR..fo..
0030 - 6f 75 58 52 22 9c 5b 08-fb ea 07 72 1c 70 88 9c  ouXR"[[...r.p..
0040 - 3d 08 ff 14 be 8d a4 80-8a bb 73 a4 76 94 12 25  =.....S.V..%
0050 - 05 91 0c 02 0f 00 90 3b-6a 07 2f 9a 55 ea 7c 53  o.i..f.id./u.$
0060 - 7c 12 a3 73 44 17 bf d0-ff 2b 91 99 52 da af 47  |..sd.....R..G
0070 - 16 a3 ef d5 d0 91 64 81-fa 88 f9 d3 1d ec 99 61  ....d.....a
0080 - 2d 0b b9 e6 d1 49 98 fb-5d 2e ec 81 fa 35 28 ac  ....t..T...5K
0090 - 6a 4b 40 92 41 c0 ed 21-81 7b f2 93 33 0f c7 e3  jK@.A..l.{..3...
00a0 - 72 74 74 d3 38 97 17 3c-93 43 56 4b e2 df 2b b6  rtt.B...c.VK..+
00b0 - 0b db a9 28 f5 46 2e 15-b2 6b 16 c1 85 73 5e 6e  ...(.F...k...s'n
00c0 - e0 da 1f c9 fd b0 25 50-bb 96 71 0e db 5d 66 b2  ....3p..q..}f.
00d0 - 9a 87 16 95 54 b9 14 76-4f d9 b9 7e 20 6b 03 e1  ....T..VO...~.k..

Start Time: 1767059087
Timeout    : 7200 (sec)
Verify return code: 18 (self-signed certificate)
Extended master secret: no
Max Early Data: 0
---
read R BLOCK
```


6.2 Infrastructure cleanup

```
module.networking.aws_internet_gateway.this: Still destroying... [id=igw-0763c7f59e0234294, 20s elapsed]
module.backend["web-2"].aws_instance.this: Destruction complete after 30s
module.backend["web-2"].aws_key_pair.this: Destroying... [id=prod-key-2]
module.backend["web-1"].aws_instance.this: Still destroying... [id=i-0190a8a47fb980f8, 30s elapsed]
module.nginx.aws_instance.this: Still destroying... [id=i-00e0adfb2e4303399, 30s elapsed]
module.backend["web-3"].aws_instance.this: Still destroying... [id=i-0fa7e70569983f786, 30s elapsed]
module.backend["web-1"].aws_key_pair.this: Destroying... [id=prod-key-1]
module.backend["web-2"].aws_key_pair.this: Destruction complete after 0s
module.backend["web-1"].aws_key_pair.this: Destruction complete after 0s
module.networking.aws_internet_gateway.this: Still destroying... [id=igw-0763c7f59e0234294, 30s elapsed]
module.nginx.aws_instance.this: Still destroying... [id=i-00e0adfb2e4303399, 40s elapsed]
module.backend["web-3"].aws_instance.this: Still destroying... [id=i-0fa7e70569983f786, 40s elapsed]
module.backend["web-3"].aws_key_pair.this: Destroying... [id=prod-key-3]
module.security.aws_security_group.backend: Destroying... [id=sg-0a409a19d2e4aff49]
module.backend["web-3"].aws_key_pair.this: Destruction complete after 0s
module.security.aws_security_group.backend: Destruction complete after 1s
module.networking.aws_internet_gateway.this: Still destroying... [id=igw-0763c7f59e0234294, 40s elapsed]
module.networking.aws_internet_gateway.this: Destruction complete after 47s
module.nginx.aws_instance.this: Still destroying... [id=i-00e0adfb2e4303399, 50s elapsed]
module.nginx.aws_instance.this: Destruction complete after 50s
module.nginx.aws_key_pair.this: Destroying... [id=prod-key-nginx]
module.networking.aws_subnet.this: Destroying... [id=subnet-058fb6ac77a3a1c1b]
module.security.aws_security_group.nginx: Destroying... [id=sg-07014c5b05d05e6e4]
module.nginx.aws_key_pair.this: Destruction complete after 0s
module.networking.aws_subnet.this: Destruction complete after 1s
module.security.aws_security_group.nginx: Destruction complete after 1s
module.networking.aws_vpc.this: Destroying... [id=vpc-086918dc88793bcd4]
module.networking.aws_vpc.this: Destruction complete after 0s

Destroy complete! Resources: 15 destroyed.
```



4. Testing Results

4.1 Load balancing tests

I verified that the Nginx reverse proxy was correctly distributing traffic between the primary web servers (Web-1 and Web-2). Using multiple browser sessions and curl requests, I observed that requests were evenly routed, and the backend servers responded as expected. The backup server (Web-3) remained idle until a failover scenario was triggered. Logs from Nginx confirmed that the upstream servers were being utilized properly.

4.2 Cache performance tests

The Nginx caching mechanism was tested by requesting the same page multiple times. The first request resulted in a cache MISS, while subsequent requests returned a cache HIT, reducing response time and load on the backend servers. Cache headers and Nginx logs were used to validate the cache behavior. Overall, caching improved the performance noticeably for repeated requests.

4.3 High availability tests

To simulate failure, I stopped one of the primary web servers (Web-1) while sending continuous requests. Nginx automatically routed traffic to Web-2, and when both primary servers were unavailable, Web-3 served as the backup. This confirmed that the system could maintain uptime during server outages, demonstrating the HA design worked as intended.

4.4 Security tests

Security groups were configured to restrict SSH access to my local IP only and allow HTTP/HTTPS traffic from anywhere to Nginx. Backend servers accepted traffic only from Nginx. I tested this by attempting direct access to the web servers from external IPs, which was blocked successfully. This confirmed that the network and server access controls were effective.

4.5 Performance Metrics:

- Response times: ~200ms for cached content, ~450ms for uncached requests.
- CPU usage: peaked at ~30% under simulated load.

- Network throughput: stable across multiple concurrent requests. These metrics indicated that the infrastructure was handling requests efficiently without overloading servers.
-

5. Challenges & Solutions

Problems Encountered:

1. **Terraform Deployment Errors:** Initial attempts failed due to misconfigured security groups and missing subnet assignments.
2. **SSH Connectivity Issues:** Nginx and backend servers were initially unreachable because my IP was not correctly captured in the Terraform variables.
3. **Nginx Configuration Mistakes:** Early Nginx setup caused caching to fail and SSL errors appeared.
4. **Failover Testing:** Web-3 did not initially serve traffic because it was not properly added to the Nginx upstream block.

Solutions:

1. Corrected Terraform scripts by properly defining VPC, subnets, and security groups with modular code.
2. Used Terraform `locals` to dynamically capture my public IP for SSH access.
3. Fixed Nginx configuration by enabling proxy caching, adding health checks, and using a self-signed SSL certificate.
4. Updated Nginx upstream configuration to include Web-3 as a backup server and verified failover by stopping primary servers.

Lessons Learned:

- Modular Terraform code improves reusability and reduces errors.
 - Proper planning of network topology and security rules is crucial before provisioning resources.
 - Automated testing of HA and caching is necessary to ensure the system works under different scenarios.
 - Small mistakes in configuration (like IP restrictions) can prevent access to critical servers.
-

6. Conclusion

This project successfully deployed a **secure, load-balanced, high-availability web infrastructure** on AWS using Terraform. I gained hands-on experience in:

- **Infrastructure as Code** with Terraform modules.
- **Web server management** with Apache and Nginx reverse proxy.
- **High availability** and failover testing.
- **Network security** using security groups.
- **Performance optimization** using caching and load balancing.

Future improvements could include: automating SSL certificates with Let's Encrypt, integrating monitoring and alerting, and extending the system with auto-scaling to handle dynamic workloads.

7. Appendices

- **Complete Code Listings:** Terraform files (`main.tf`, `modules/*`), server bootstrap scripts (`apache-setup.sh`, `nginx-setup.sh`).
 - **Configuration Files:** Nginx config (`nginx.conf`), Apache config (`httpd.conf`), security group rules.
 - **Additional Screenshots:** Load balancing tests, cache HIT/MISS logs, failover demonstration, Terraform output.
 - **References:** AWS documentation, Terraform documentation, Nginx official guides.
-

Repo Link: <https://github.com/hajrasarwar11/assignment2>